

RENEWABLY.IE

Production Deployment and Configuration Guide

Developer Technical Documentation

Version 1.0 | April 2026

This guide covers everything needed to deploy the Renewably CRM to production, including environment configuration, Supabase database setup, third-party service integration (Postmark, Stripe, Google Calendar), Docker deployment, security hardening, and a comprehensive go-live checklist for the engineering team.

Table of Contents

1. Architecture Overview
2. Environment Variables Reference
3. Supabase Database Setup
4. Postmark Email Configuration
5. Stripe Billing Configuration
6. Google Calendar Integration
7. Redis Configuration (Optional)
8. Build and Deployment
9. Docker Deployment
10. Security Configuration
11. Rate Limiting and Performance
12. Monitoring and Logging
13. Go-Live Checklist
14. Rollback Procedures
15. Troubleshooting Common Issues

1. Architecture Overview

The Renewably CRM is a full-stack Next.js 16 application built with React 19, TypeScript 5, Prisma ORM, and Tailwind CSS 4. The application runs in standalone output mode, which produces a self-contained server bundle suitable for Docker containerisation. In development, the project uses SQLite as the database backend via Prisma for rapid prototyping. For production, the application connects to a Supabase PostgreSQL instance with Row-Level Security (RLS) policies, providing enterprise-grade data isolation and security.

The platform integrates with four external services: Supabase (database and auth), Postmark (transactional email), Stripe (subscription billing), and Google Calendar (scheduling). Redis is optionally used for distributed rate limiting and session caching, with an in-memory fallback for deployments without Redis infrastructure. The AI chatbot feature uses the z-ai-web-dev-sdk for conversational AI capabilities and automatic lead capture from website visitors.

1.1 Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js (App Router)	16.x	Full-stack web application framework
Frontend	React	19.x	UI component library
Language	TypeScript	5.x	Type-safe JavaScript
Database (Dev)	SQLite + Prisma	6.x	Local development database
Database (Prod)	Supabase PostgreSQL	15+	Production database with RLS
ORM	Prisma	6.x	Database client and migrations
Styling	Tailwind CSS	4.x	Utility-first CSS framework
UI Library	shadcn/ui + Radix	Latest	Accessible component primitives
Email	Postmark (REST API)	-	Transactional email delivery
Billing	Stripe	22.x	Subscription payments
Calendar	Google Calendar API	-	Meeting scheduling and sync
Cache	Redis (ioredis)	5.x	Rate limiting and session caching
Runtime	Bun	Latest	Production JavaScript runtime
AI	z-ai-web-dev-sdk	-	AI chatbot and lead capture

Table 1: Complete technology stack

1.2 Request Flow

Incoming requests pass through the Next.js middleware (src/middleware.ts) which applies rate limiting to the login endpoint and logs all CRM API requests with timing information. Authenticated requests are validated via session cookies against the sessions table. API routes handle business logic, Zod schema validation, and database operations via Prisma. Response times are tracked via the X-Response-Time header. All API responses follow a consistent JSON format with structured error messages including field-level validation details when appropriate.

2. Environment Variables Reference

The application requires 17 environment variables for full production operation. These variables control database connectivity, email sending, payment processing, calendar integration, and AI features. Each variable is documented below with its purpose, format, and whether it is required for production deployment. Variables marked as optional have sensible defaults or fallback behaviour that allows the application to function without them.

2.1 Complete Variable Listing

Variable	Required	Default	Service	Description
DATABASE_URL	Yes	(none)	Supabase	PostgreSQL connection string (postgresql://user:pass@host:5432/db)
POSTMARK_SERVER_TOKEN	Yes	(none)	Postmark	API token for email sending
FROM_EMAIL	No	hello@renewably.ie	Postmark	Sender email address
STRIPE_SECRET_KEY	Yes	(none)	Stripe	Secret key for payment processing
STRIPE_WEBHOOK_SECRET	Yes	(none)	Stripe	Webhook signing secret for verifying events
STRIPE_PRICE_STARTER	Yes	(none)	Stripe	Price ID for the Starter subscription plan
STRIPE_PRICE_PRO	Yes	(none)	Stripe	Price ID for the Professional subscription plan
STRIPE_PRICE_ENTERPRISE	Yes	(none)	Stripe	Price ID for the Enterprise subscription plan
GOOGLE_CLIENT_ID	No	(none)	Google	OAuth client ID for Calendar integration
GOOGLE_CLIENT_SECRET	No	(none)	Google	OAuth client secret for Calendar integration

NEXT_PUBLIC_APP_URL	Yes	(none)	Core	Public URL of the app (e.g. https://app.renewably.ie)
AGENT_API_KEY	No	(none)	Core	API key for agent content management endpoints
REDIS_URL	No	redis://localhost:6379	Redis	Redis connection URL for rate limiting
NODE_ENV	Yes	development	Core	Set to "production" for production builds
LOG_LEVEL	No	info	Core	Logging level: debug, info, warn, error
ANTHROPIC_API_KEY	No	(none)	AI	Claude API key (for website analytics AI check)

Table 2: Complete environment variable reference

Critical: Never commit environment variables to version control. Use .env.local for local development (already in .gitignore) and your hosting platform dashboard or secrets manager for production. Rotate secrets periodically and audit access logs.

2.2 Production .env.example

```
# — Database —  
DATABASE_URL=postgresql://crm_user:YOUR_PASSWORD@db.renewably.supabase.co:5432/postgres  
# — Email (Postmark) —  
POSTMARK_SERVER_TOKEN=xxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  
FROM_EMAIL=hello@renewably.ie  
# — Billing (Stripe) —  
STRIPE_SECRET_KEY=sk_live_xxxxxxxxxxxxxxxxxx  
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxxxxxx  
STRIPE_PRICE_STARTER=price_xxxxxxxxxxxxxxxxxx  
STRIPE_PRICE_PRO=price_xxxxxxxxxxxxxxxxxx  
STRIPE_PRICE_ENTERPRISE=price_xxxxxxxxxxxxxxxxxx  
# — Google Calendar —  
GOOGLE_CLIENT_ID=xxxx.apps.googleusercontent.com  
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxx  
# — App —  
NEXT_PUBLIC_APP_URL=https://app.renewably.ie  
NODE_ENV=production  
LOG_LEVEL=info  
# — Optional —  
REDIS_URL=redis://your-redis:6379  
AGENT_API_KEY=your-agent-api-key  
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxx
```

3. Supabase Database Setup

The production database runs on Supabase (managed PostgreSQL 15+). The complete SQL schema is provided in supabase-schema.sql in the project download directory. This section covers the step-by-step process for creating the Supabase project, applying the schema, and configuring Row-Level Security policies for data isolation.

3.1 Step-by-Step Setup

Step	Action	Details
1	Create Supabase project	Go to supabase.com, create a new project, select EU-West-1 (Ireland) region for latency
2	Note the connection string	Project Settings > Database > Connection string (URI format). Replace [YOUR-PASSWORD]

3	Set DATABASE_URL	Use the connection string as your DATABASE_URL environment variable
4	Apply the schema	Run supabase-schema.sql in the SQL Editor (supabase.com > SQL Editor > New query)
5	Verify tables	Run: SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'
6	Enable RLS	Uncomment the RLS policy blocks in supabase-schema.sql and apply them
7	Create a service role key	Project Settings > API > service_role key (bypasses RLS for server-side ops)
8	Test connectivity	Run the app with DATABASE_URL set and verify the login endpoint works

Table 3: Supabase setup steps

3.2 Schema Highlights

The production schema (supabase-schema.sql) includes 17 PostgreSQL enum types, 30+ tables, automated triggers for computed columns (invoice totals, auto-balance on payment), and Row-Level Security policy stubs ready for activation. Key design decisions include UUID primary keys (replacing SQLite CUID strings), JSONB columns for flexible metadata storage, and a polymorphic entity_tags table that serves all entity types through a single junction table rather than separate join tables per entity. The schema also includes seed data for default application settings and an initial admin user account.

Prisma Migration: When migrating from SQLite (dev) to Supabase PostgreSQL (prod), you must update the Prisma schema provider from "sqlite" to "postgresql" and regenerate the Prisma client with: `npx prisma generate`. The connection string format changes from `file:path` to a full PostgreSQL URI. Review `prisma/schema.prisma` and update any SQLite-specific syntax (e.g., `DateTime` defaults).

4. Postmark Email Configuration

Postmark handles all transactional email delivery for the CRM. The complete Postmark integration reference is provided in the companion document "Renewably_Postmark_API_Reference.pdf". This section covers the production-specific setup steps that must be completed before emails can be sent to real customers.

Step	Action	Details
1	Create Postmark account	Sign up at postmarkapp.com (free tier: 100 emails/month)
2	Add sending domain	Add renewably.ie in Settings > Domains. Postmark provides DNS records to add
3	Configure DNS	Add SPF, DKIM, and DMARC records to your domain DNS. Postmark verifies automatically
4	Verify sender signature	Add hello@renewably.ie as a sender signature. Confirm via email verification

5	Create server token	Servers > Your Server > API Tokens > Generate. Copy to POSTMARK_SERVER_TOKEN
6	Test with sandbox	Use Postmark sandbox mode to test email rendering before going live
7	Set up webhooks	Configure delivery, bounce, open, and click webhooks (see Postmark API Reference)
8	Enable alerts	Set up Postmark alerts for bounce rate spikes and delivery failures

Table 4: Postmark production setup steps

5. Stripe Billing Configuration

Stripe powers the installer subscription billing system. The CRM uses Stripe Checkout Sessions for subscription creation, the Customer Portal for self-service billing management, and webhooks for real-time subscription event processing. The Stripe integration is implemented in `src/lib/stripe.ts` using the official Stripe Node.js SDK with lazy singleton instantiation and API version pinning (2025-04-30.basil).

5.1 Setup Steps

Step	Action	Details
1	Create Stripe account	stripe.com. Switch to Live mode for production
2	Create 3 products	Create Starter, Professional, and Enterprise products with monthly pricing
3	Copy Price IDs	Each product has a Price ID (price_XXX). Set as STRIPE_PRICE_STARTER/PRO/ENTERPRISE
4	Get API keys	Dashboard > Developers > API keys. Copy the Secret key to STRIPE_SECRET_KEY
5	Configure webhook	Developers > Webhooks > Add endpoint. URL: https://app.renewably.ie/api/crm/billing/webhook
6	Select webhook events	Enable: checkout.session.completed, customer.subscription.updated, customer.subscription.deleted, invoice.payment_failed, invoice.payment_succeeded
7	Copy webhook secret	The webhook signing secret (whsec_XXX) goes to STRIPE_WEBHOOK_SECRET
8	Enable Customer Portal	Stripe Settings > Billing > Customer portal. Configure allowed features

Table 5: Stripe production setup steps

Webhook Security: The billing webhook endpoint (POST `/api/crm/billing/webhook`) is intentionally excluded from authentication middleware since it is called directly by Stripe servers. All webhook payloads are verified using the `STRIPE_WEBHOOK_SECRET` via the `stripe.webhooks.constructEvent()` method. Never skip webhook verification in production.

6. Google Calendar Integration

The CRM integrates with Google Calendar to sync meetings between the CRM and the installer's Google Calendar. The integration uses OAuth 2.0 for authentication and the Google Calendar API for reading and writing calendar events. This integration is optional; if `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` are not set, the calendar endpoints operate in mock mode, returning empty results and allowing the rest of the CRM to function normally.

6.1 Setup Steps

Step	Action	Details
1	Google Cloud Console	Go to console.cloud.google.com , create a project (or select existing)
2	Enable Calendar API	APIs & Services > Library > search "Google Calendar API" > Enable
3	Configure OAuth consent	APIs & Services > OAuth consent screen. Fill in app name, support email
4	Create OAuth credentials	APIs & Services > Credentials > Create OAuth 2.0 Client ID > Web application
5	Add redirect URI	Add: https://app.renewably.ie/api/crm/calendar/google/callback
6	Copy credentials	Client ID to <code>GOOGLE_CLIENT_ID</code> , Client Secret to <code>GOOGLE_CLIENT_SECRET</code>
7	Test OAuth flow	Trigger the auth URL endpoint in CRM, complete the OAuth flow in browser

Table 6: Google Calendar OAuth setup steps

7. Redis Configuration (Optional)

Redis is used for distributed rate limiting on public API endpoints (contact form, AI chat) and can be used for session caching. The implementation in `src/lib/redis.ts` uses `ioredis` with a lazy connection strategy, meaning Redis is only connected to when first needed. If Redis is unavailable or not configured, the system automatically falls back to in-memory rate limiting using a JavaScript Map. This means the application works without Redis, but rate limits reset on server restart since in-memory state is not persisted.

For production deployments with multiple server instances (horizontal scaling), Redis becomes essential because in-memory rate limiting is per-process and cannot coordinate across instances. Without Redis, each server instance maintains its own rate limit counters, effectively multiplying the allowed request rate by the number of running instances. Recommended Redis providers include Upstash (serverless), Redis Cloud, or a self-hosted Redis instance on the same network as the application server.

8. Build and Deployment

The application uses Next.js standalone output mode, which produces a self-contained deployment bundle that includes the server runtime, all dependencies, and static assets. The build process creates the `.next/standalone` directory which can be deployed directly without installing `node_modules` on the target server.

8.1 Build Commands

```
# Install dependencies  
npm install  
  
# Generate Prisma client (after  
updating schema provider to postgresql)  
npx prisma generate  
  
# Build the  
application (standalone output mode)  
npm run build  
  
# This runs: next build &&  
cp -r .next/static .next/standalone/.next/ && cp -r public  
.next/standalone/  
  
# Start the production server  
npm run start  
  
# This  
runs: NODE_ENV=production bun .next/standalone/server.js  
  
# The server starts  
on port 3000 by default
```

8.2 Important Build Notes

TypeScript Build Errors: The current `next.config.ts` has `typescript.ignoreBuildErrors` set to `true`. This MUST be set to `false` before production deployment. Fix all TypeScript errors first, then deploy. Running with `ignoreBuildErrors:true` in production masks potential runtime type bugs.

Setting	Current	Production Required	File
<code>typescript.ignoreBuildErrors</code>	<code>true</code>	<code>false</code>	<code>next.config.ts</code>
<code>output</code>	<code>"standalone"</code>	<code>"standalone"</code>	<code>next.config.ts</code>
<code>reactStrictMode</code>	<code>true</code>	<code>true</code>	<code>next.config.ts</code>
<code>poweredByHeader</code>	<code>false</code>	<code>false</code>	<code>next.config.ts</code>

Table 7: Build configuration checklist

9. Docker Deployment

The standalone output mode is ideal for Docker containerisation. The following Dockerfile and docker-compose configuration provide a production-ready deployment setup. The Docker image uses a multi-stage build to minimise the final image size, copying only the standalone output and static assets into the production stage.

9.1 Dockerfile

```
# — Stage 1: Build —  
FROM node:20-alpine AS builder  
WORKDIR /app  
COPY package.json bun.lockb* ./  
RUN npm install  
COPY . .  
RUN npx prisma generate  
RUN npm run build  
  
# — Stage 2: Production —  
FROM node:20-alpine AS runner  
WORKDIR /app  
ENV NODE_ENV=production  
COPY --from=builder /app/.next/standalone ./  
COPY --from=builder /app/.next/static ./  
COPY --from=builder /app/public ./public  
EXPOSE 3000  
CMD ["node", "server.js"]
```

9.2 Docker Compose (with Redis)

```
version: "3.9"<br/>services:<br/> app:<br/> build: .<br/> ports:<br/> -<br/> "3000:3000"<br/> env_file: .env.production<br/> depends_on:<br/> - redis<br/> restart:<br/> unless-stopped<br/><br/> redis:<br/> image: redis:7-alpine<br/> ports:<br/> -<br/> "6379:6379"<br/> volumes:<br/> - redis_data:/data<br/> restart:<br/> unless-stopped<br/><br/>volumes:<br/> redis_data:
```

10. Security Configuration

The application implements multiple layers of security including HTTP security headers, session-based authentication with HTTP-only cookies, input validation via Zod schemas, rate limiting on sensitive endpoints, and webhook signature verification. This section documents the security configuration that must be verified before production deployment.

10.1 Security Headers

The following security headers are configured in `next.config.ts` via the `headers()` function. These headers are applied to all routes and provide protection against clickjacking, cross-site scripting, MIME type sniffing, and other common web vulnerabilities. Verify these headers are present in the production deployment using `curl -I` or browser developer tools.

Header	Value	Purpose
X-Frame-Options	SAMEORIGIN	Prevents clickjacking by disallowing framing from other origins
X-Content-Type-Options	nosniff	Prevents MIME type sniffing by the browser
Referrer-Policy	strict-origin-when-cross-origin	Controls referrer information sent with requests
X-XSS-Protection	1; mode=block	Enables browser XSS filter (legacy, superseded by CSP)
Permissions-Policy	camera=(), microphone=(), geolocation=()	Disables camera, mic, and location access
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Forces HTTPS for 1 year, includes subdomains
Content-Security-Policy	default-src 'self'; ...	Controls resource loading (see note below)

Table 8: Security headers configured in `next.config.ts`

CSP Note: The current Content-Security-Policy includes 'unsafe-inline' and 'unsafe-eval' for script-src to support Next.js runtime requirements. For production, review the CSP and restrict these to specific nonces or hashes where possible. The current policy is functional but could be tightened for enhanced XSS protection.

10.2 Authentication Security

Feature	Implementation	Configuration
Session storage	Database-backed sessions table	7-day expiry, HTTP-only cookie
Cookie flags	HttpOnly, SameSite=Lax, Secure (prod)	Secure flag auto-set when NODE_ENV=production
Password hashing	bcryptjs with cost factor 12	12 salt rounds (industry standard)
Session token	crypto.randomBytes(32).toString("hex")	256-bit cryptographically secure random token
Login rate limit	5 attempts per minute per IP	Configurable in src/lib/crm-session.ts
Admin protection	requireAdmin() checks role === "admin"	Applied to settings and sensitive routes

Table 9: Authentication security configuration

11. Rate Limiting and Performance

Rate limiting is implemented at two levels: middleware-level for the login endpoint (in-memory, 10 requests per minute per IP) and application-level for public API endpoints (Redis-backed with in-memory fallback). The application-level rate limiter supports configurable windows and limits via the `checkApiRateLimit()` function in `src/lib/crm-validation.ts`.

11.1 Rate Limit Configuration

Endpoint	Limit	Window	Backend	Config File
POST /api/crm/auth/login	10 req	60 sec	In-memory (middleware)	src/middleware.ts
Login (app-level)	5 attempts	60 sec	In-memory	src/lib/crm-session.ts
POST /api/contact	5 req	15 min	Redis + in-memory	src/lib/rate-limit.ts
POST /api/chat	20 req	15 min	Redis + in-memory	src/lib/rate-limit.ts
CRM API routes	No global limit	-	None	-

Table 10: Rate limit configuration by endpoint

Production Warning: There is no global rate limit on authenticated CRM API routes. In production, consider adding a per-user rate limit (e.g., 100 requests per minute) to prevent abuse from compromised accounts. This is especially important if the CRM is exposed to the internet without a VPN or IP whitelist restriction.

12. Monitoring and Logging

The application uses a structured JSON logger (`src/lib/logger.ts`) that outputs timestamped log entries with level, message, and optional metadata. The log level is configurable via the `LOG_LEVEL` environment

variable, defaulting to "info" in production. All CRM API requests are logged by the middleware with timestamp, method, and path. Response times are measured and added via the X-Response-Time response header.

12.1 Log Levels

Level	Use Case	Production Setting
debug	Detailed development diagnostics, variable values, query logging	OFF in production
info	Request logging, email sends, user actions, normal operations	ON (default)
warn	Non-critical issues: missing config, fallbacks activated, retries	ON
error	API failures, database errors, authentication failures, unhandled exceptions	ON

Table 11: Log level configuration

12.2 Recommended Monitoring Stack

For production monitoring, the following tools are recommended: (1) Application Performance Monitoring (APM) via Datadog or New Relic for request latency tracking and error alerting; (2) Uptime monitoring via UptimeRobot or BetterUptime to detect downtime; (3) Log aggregation via Logtail, Papertrail, or Loki to centralise logs from all instances; (4) Error tracking via Sentry to capture unhandled exceptions with stack traces and user context; (5) Database monitoring via Supabase dashboard for query performance, connection pooling, and storage usage. Each tool should be configured with appropriate alert thresholds and notification channels (Slack, email, PagerDuty) to ensure the team is alerted to issues before users notice.

13. Go-Live Checklist

This comprehensive checklist covers every task that must be completed before the CRM can go live in production. Each item is categorised by criticality: Critical items must be completed before launch, High items should be completed before launch but can be deferred by 1-2 days, and Medium items can be addressed in the first week after launch. Use this checklist during the deployment sprint to track progress.

13.1 Infrastructure

#	Task	Priority
1	Set NODE_ENV=production in all environments	Critical
2	Set typescript.ignoreBuildErrors=false in next.config.ts and fix all errors	Critical
3	Deploy behind HTTPS with valid SSL certificate	Critical
4	Configure DNS: A/CNAME records for app.renewably.ie	Critical
5	Set up reverse proxy (Nginx/Caddy) or use platform managed HTTPS	Critical

6	Configure static asset caching (images: 86400s, JS/CSS: content-hash)	High
7	Set up health check endpoint (GET / returns 200)	High
8	Configure container auto-restart policy (restart: unless-stopped)	High

Table 12: Infrastructure go-live checklist

13.2 Database

#	Task	Priority
1	Apply supabase-schema.sql to production Supabase project	Critical
2	Verify all 30+ tables exist with correct schema	Critical
3	Enable Row-Level Security policies	Critical
4	Update Prisma schema provider to "postgresql"	Critical
5	Run npx prisma generate with PostgreSQL provider	Critical
6	Create database backup schedule (daily automated)	High
7	Verify connection pooling settings in Supabase	High
8	Seed initial admin user and default settings	Critical

Table 13: Database go-live checklist

13.3 Third-Party Services

#	Task	Priority
1	Set POSTMARK_SERVER_TOKEN (production token, not sandbox)	Critical
2	Verify sending domain DNS records (SPF, DKIM, DMARC)	Critical
3	Set STRIPE_SECRET_KEY (live mode, not test mode)	Critical
4	Set STRIPE_PRICE_STARTER/PRO/ENTERPRISE (live Price IDs)	Critical
5	Configure Stripe webhook endpoint and verify with stripe listen	Critical
6	Set STRIPE_WEBHOOK_SECRET	Critical
7	Set GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET (if calendar integration needed)	High
8	Set NEXT_PUBLIC_APP_URL to production URL	Critical
9	Set REDIS_URL if using distributed rate limiting	High
10	Test all email flows: contact form, welcome, proposal, invoice	Critical
11	Test Stripe checkout flow end-to-end	Critical

12	Test Google Calendar OAuth flow (if enabled)	Medium
----	--	--------

Table 14: Third-party services go-live checklist

13.4 Security

#	Task	Priority
1	Verify all security headers are present (curl -I)	Critical
2	Verify HTTPS redirects HTTP traffic	Critical
3	Change default admin password after first login	Critical
4	Audit all API routes for proper auth guards	Critical
5	Verify session cookies are HttpOnly, Secure, SameSite=Lax	Critical
6	Test rate limiting on login endpoint (10 req/min)	High
7	Test rate limiting on contact form (5 req/15min)	High
8	Verify Stripe webhook signature verification	Critical
9	Remove or disable any debug endpoints	High
10	Review and restrict CORS origins if needed	Medium

Table 15: Security go-live checklist

14. Rollback Procedures

In the event of a failed deployment or critical production issue, the following rollback procedures should be followed. The key principle is to minimise downtime and data loss. Since the application state is primarily in the Supabase database (which is managed externally), rollback of the application code does not affect data. However, database schema changes must be handled carefully as they may not be backward-compatible with previous application versions.

14.1 Application Rollback

If a deployment introduces a critical bug, rollback to the previous Docker image or git commit. With Docker, this means re-deploying the previous image tag. With a git-based deployment, revert to the previous commit, rebuild, and redeploy. The standalone output mode means there are no external runtime dependencies beyond the environment variables, making rollback straightforward. Always tag Docker images with the git commit hash or semantic version for easy rollback.

14.2 Database Rollback

Database schema changes in Supabase should be applied as reversible migrations. The supabase-schema.sql file uses CREATE TABLE IF NOT EXISTS statements, making it idempotent for new deployments.

However, if a migration adds or modifies columns, a rollback migration must be prepared that reverses those changes. Always test schema changes in a staging Supabase project before applying to production. Enable Supabase Point-in-Time Recovery (PITR) for the ability to restore the database to any moment in the last 7 days, which serves as the ultimate rollback mechanism for data-level issues.

15. Troubleshooting Common Issues

This section documents the most common issues encountered during production deployment and operation, along with their causes and solutions. Use this section as a first reference when diagnosing problems.

Issue	Likely Cause	Solution
Login returns 500	Database connection failure	Verify DATABASE_URL, check Supabase status, test connectivity
Emails not sending	Missing POSTMARK_SERVER_TOKEN	Set the env var, verify token in Postmark dashboard
Stripe checkout fails	Wrong Price ID or test mode keys	Verify STRIPE_PRICE_* are live Price IDs, check STRIPE_SECRET_KEY
Webhook not receiving events	Incorrect URL or secret	Verify endpoint URL, check STRIPE_WEBHOOK_SECRET matches
Build fails with TS errors	ignoreBuildErrors:false	Fix TypeScript errors or temporarily set true for emergency deploy
Session not persisting	Cookie not set or HttpOnly issue	Verify Secure flag is correct for HTTP/HTTPS, check cookie path
Rate limit triggered too fast	Multiple server instances without Redis	Deploy Redis or switch to single instance for rate limiting
Google Calendar not connecting	Missing OAuth credentials	Set GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET env vars
Static assets 404	Missing copy step in build	Verify the build script copies .next/static and public to standalone/
HSTS preload warning	Strict-Transport-Security too aggressive	Reduce max-age or remove preload for initial deployment

Table 16: Common production issues and solutions

15.1 Useful Diagnostic Commands

```
# Check if the app is responding<br>curl -I https://app.renewably.ie/<br><br># Verify security headers<br>curl -sI https://app.renewably.ie/ | grep -i "x-frame\|x-content\|strict-transport"<br><br># Test database connectivity from the server<br>psql $DATABASE_URL -c "SELECT 1;"<br><br># Check Docker container logs<br>docker logs renewably-crm --tail 100 -f<br><br># Test Postmark token validity<br>curl -s -X POST https://api.postmarkapp.com/email/withTemplate \<br> -H "X-Postmark-Server-Token: $POSTMARK_SERVER_TOKEN" \<br> -H "Content-Type:
```

```
application/json" \<br/> -d '{"From":"test@renewably.ie","To":"test@renewably.ie","Subject":"Test","HtmlBody":"<p>OK</p>"}
```